

# Appendix B: ORAMA chain design

Written 3 October 2026 against branch 1.0.0 of the Orama repository. Every statement describes what the code on that branch does, unless it is labeled "Designed, not yet built" or "Open decision." Nothing described here runs on a public network.

## 1. Summary

---

ORAMA is the native asset of the Orama L1, a proof-of-stake blockchain with three purposes:

1. **Pay the people who run the network.** Validators, storage providers and relay operators are paid from a fixed, public emission schedule with no pre-allocation to anyone.
2. **Private payments by default.** User-to-user transfers of ORAMA are possible only through a shielded pool. There is no public path between users.
3. **An app ecosystem.** Contracts, user tokens and compressed NFTs run on the chain, and each app's full backend (database, storage, functions) runs on Orama's cloud.

Four properties define the design:

- **Zero premine.** Code rejects any genesis with nonzero supply. There is no founder, team, investor, sale, airdrop or treasury allocation. Every ORAMA is earned by running infrastructure.
- **No admin keys.** No module has an owner, pause, freeze, blacklist or halt function.
- **Capped power.** No validator holds more than 5% of voting power (3% above 60 validators), whatever its stake.
- **Everything ships in the genesis binary.** Where a feature must wait for something, its activation rule is written into the code.

The Orama company will run nodes and validators from genesis and earn rewards under the same rules as any other operator.

Current state: the chain is a Go module of about 55,000 non-test, non-generated lines plus about 7,000 lines of Rust, with roughly 1,260 Go test functions in 242 test files. It has run on a private multi-node test network ("stagenet"). It has had no external audit, no incentivized testnet and has no mainnet date. Section 10 gives status by module.

## 2. Engine and accounts

---

**Stack.** Cosmos SDK v0.54.4, CometBFT v0.39.4 and CosmWasm (wasmd v0.70.3 on wasmvvm v3.0.7), in Go, with Rust libraries linked for the contract VM and the shielded-proof verifier. One static binary, `oramad`, runs under `cosmovisor` for coordinated upgrades.

**Why this engine.** The project's plan (decision D6) records a comparison of about fifteen alternatives, including Algorand, Aleo, Penumbra, Mina, NEAR, Sui, Substrate and a Solana fork. The reasoning:

- BFT consensus with instant finality, no proof of work, and a mature validator ecosystem.

- Emission, power and governance are written as native Go modules, so they are part of the state machine and cannot be upgraded around.
- CosmWasm for contracts. The plan rejects the Cosmos EVM module (it cites five critical advisories in 2026) and rejects running two virtual machines. A Solana fork was judged the heaviest option for a small team, with no native path to the custom modules.
- No IBC and no bridges, in this design (D25). That removes a large attack surface.

The three research reports the plan cites are not in the repository, so this summary comes from the plan's decision table, not from the reports.

#### **Accounts.**

- Base denomination `norama`, display denomination `ORAMA`, nine decimals (1 ORAMA = 10<sup>9</sup> norama), address prefix `orama`.
- Keys are compressed secp256k1, not Ethereum-style keys. Transactions use `SIGN_MODE_DIRECT` (and textual mode where the client is online).
- BIP-44 coin type is 118, a placeholder (Open decision).
- Validators sign with ed25519. Each storage or relay node also has a secp256k1 "hot key" that holds only a small fee balance and cannot move operator funds.

### **3. Consensus, block time and finality**

---

**Configuration.** Orama does not override CometBFT's consensus timeouts. A new node inherits the Cosmos SDK behavior of writing `timeout_commit = 5s` into its configuration (SDK `server/util.go`; CometBFT's own default is 1 s). Other CometBFT defaults apply.

**Measured.** The fleet end-to-end suite recorded block headers on a three-validator chain spread across hosts on 2 October 2026 (commit 2cf8b444). Consecutive blocks were 5.2 to 5.3 seconds apart; the run averaged 5.28 s over 3,606 blocks, including restart tests. This is a three-validator measurement. The planned benchmark at 30 to 150 validators across regions (spike C0-2) has not been run.

**Finality.** CometBFT commits a block once more than two thirds of voting power has precommitted it. That block is final: there is no confirmation depth and no reorganization, unless more than one third of power signs conflicting blocks, which is slashable (5% of stake and permanent removal). The fleet suite checked that all validators agreed on every block and application hash in a sampled window, and that with three equal seats the chain halts rather than commits when one is stopped, since exactly two thirds is not a quorum.

**Comparison.** Figures collected 3 October 2026. "Secondary" marks sources I did not check against validator telemetry.

| Chain                          | Block or slot time | Finality                                             | Source                                                                                                                                                                                      |
|--------------------------------|--------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Orama (measured, 3 validators) | about 5.2 s        | at commit                                            | Orama e2e artifact, 2 Oct 2026                                                                                                                                                              |
| Cosmos Hub                     | about 6 s          | at commit                                            | <a href="https://ryder.id/blogs/post/cosmos-atom-in-2026-ibc-and-the-interchain-ecosystem">https://ryder.id/blogs/post/cosmos-atom-in-2026-ibc-and-the-interchain-ecosystem</a> (secondary) |
| Celestia                       | 6 s (3 s proposed) | at commit                                            | <a href="https://github.com/celestiaorg/celestia-app/issues/6488">https://github.com/celestiaorg/celestia-app/issues/6488</a> (30 Jan 2026)                                                 |
| Osmosis                        | about 1.5 s        | at commit                                            | <a href="https://www.bitget.com/news/detail/12560604068637">https://www.bitget.com/news/detail/12560604068637</a> (secondary)                                                               |
| dYdX v4                        | about 1 s          | at commit                                            | <a href="https://docs.dydx.xyz/concepts/onboarding-faq">https://docs.dydx.xyz/concepts/onboarding-faq</a> (secondary)                                                                       |
| Injective                      | about 0.65 s       | at commit                                            | <a href="https://staking-explorer.com/explorer/injective">https://staking-explorer.com/explorer/injective</a> (secondary)                                                                   |
| Cosmos stack roadmap           | 500 ms target      | at commit                                            | <a href="https://x.com/cosmos/status/2011830787491185071">https://x.com/cosmos/status/2011830787491185071</a> (Jan 2026)                                                                    |
| Solana                         | about 0.4 s slots  | about 12.8 s; Alpenglow targets 100 to 150 ms        | <a href="https://www.helius.dev/blog/alpenglow">https://www.helius.dev/blog/alpenglow</a>                                                                                                   |
| Ethereum                       | 12 s slots         | about 12.8 minutes; single-slot finality is research | <a href="https://medium.com/@0s.and.1s/ethereum-101-finality-explained-8f4c8b18d94b">https://medium.com/@0s.and.1s/ethereum-101-finality-explained-8f4c8b18d94b</a> (secondary)             |

After a block gathers its precommits, CometBFT waits `timeout_commit` before the next height. It is the floor on block time on a healthy network. Fast Cosmos chains lower it and accept more sensitivity to network latency, and larger validator sets spread over more regions lengthen rounds. I found no citable source quantifying validator count against latency. Orama's interval matches the 5 s setting. Operators can lower it locally, and the 30-plus-validator benchmark should inform that choice.

**Calibration (Open decision).** Some parameters count blocks on an assumed 5 to 6 s: validator history (201,600 blocks, written as 14 days at 6 s), the shielded anchor window (14,400 blocks, written as 24 hours at 6 s) and the contract upload sunset (3,162,240 blocks, written as 183 days at 5 s). At the measured 5.2 to 5.3 s these are about 12 days, 21 hours and 190 days. The emission epoch is unaffected, because it closes on 24 hours of block time plus at least 14,400 blocks. These should be settled before genesis.

**Line for a deck.** "Blocks about every 5 seconds, final the moment they commit: no reorgs, no waiting for confirmations." It is measured on stagenet with three validators. Orama is not competing on raw speed.

## 4. Token economics

---

**Supply schedule.** No ORAMA exists at genesis. The chain mints on 24-hour epochs. An epoch closes only when both 24 hours of block time and 14,400 blocks have passed, and a gap is never caught up: after a halt, exactly one epoch closes.

| Epochs         | Approx. years | Maximum per epoch |
|----------------|---------------|-------------------|
| 1 to 730       | 1 to 2        | 14,848 ORAMA      |
| 731 to 1,460   | 3 to 4        | 7,424             |
| 1,461 to 2,190 | 5 to 6        | 3,712             |
| 2,191 to 2,920 | 7 to 8        | 1,856             |
| 2,921 to 3,650 | 9 to 10       | 928               |
| 3,651 onward   | 11 on         | 274, forever      |

The cumulative maximum at epoch 3,650 is exactly 21,000,640 ORAMA (pinned by a unit test). The tail adds about 100,000 ORAMA per year ( $274 \times 365 = 100,010$ ). There is no terminal cap. The schedule is a maximum, not a promise, and the table and tail are ossified: no message can change them.

**Emission split.** Each epoch's maximum divides 60% validators and delegators, 25% storage, 10% relay and 5% development. Only the validator share is minted at epoch close. Storage and relay shares are ceilings, minted lazily and only up to the ceiling when a payment is earned. The development share is minted only when both governance houses approve a spend, and only after governance opens. After governance opens, each share may move up to 10 points from canonical by two-house vote.

**Zero premine, in code.** Genesis rejects any nonzero supply unless a devnet-only flag is set, and that flag also requires a devnet, stagenet or localnet chain identifier. A locked-genesis check refuses to start a production-style chain whose parameters differ from the signed values. Bootstrap validators receive voting power, not tokens, and the code has no allocation path for a founder, team, investor or treasury. The invariant `supply = genesis supply + minted - burned` is checked at every genesis load and in a simulated 4,000-epoch run spanning every halving.

**Fees.** Each transaction pays a base fee per unit of gas. It adjusts block by block toward 50% fullness, by at most 12.5% per block, above a floor (placeholder: 1 norama per gas). Anything paid above it is a tip to the block proposer, paid from a bank balance only. There is no ordering auction; ordering is first in, first out within a fee tier.

*Current implementation: the base fee is burned in full.* Smaller burns also exist: 1% of every released state deposit, the token creation fee, a per-nullifier fee on shielded actions, and 5% of storage payments.

*Open decision before mainnet: the destination of the base fee.* The founder's current view is that the base fee probably should be kept, not burned, with the destination undecided. Options:

| Option                                    | Trade-offs                                                                                                                                                                    |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Keep the burn                             | Simple, nothing to capture, supply-reducing. Operators and developers get no fee income, so funding rests on emission alone.                                                  |
| Route to validators and operators         | Income grows with usage and offsets the shrinking emission. Must be distributed by capped power, or fees favor large validators.                                              |
| Protocol treasury spent by the two houses | Sustainable funding for audits and maintenance. A standing balance is a capture target, so it needs a keyless account and spending only through the timelocked house process. |
| Development fund                          | Replaces minting for the development share. Same capture concern; the 5% development ceiling would need re-examination.                                                       |

Two constraints apply to every option. First, the burn rule is on the ossified list: a test confirms governance messages have no field that could touch it. The destination cannot be changed by vote after launch, only by a hard fork, so it must be decided and written into the genesis binary. Second, it must stay consistent with "no admin keys": a protocol rule or a keyless account spent only through the house process. Any change also rewrites the supply invariant and fee tests, so it is a code change, not a parameter.

**Earnings accounts.** Every protocol payout (emission, tips, storage and relay payments, development spends) lands in the recipient's earnings account, not a public balance. Earnings can pay the holder's own transaction fees, bonds, deposits and token fees, and can move into the shielded pool or a node's fee-only hot-key balance. They cannot be sent to another user. This is why an operator can start with a zero balance and buy nothing first.

**State deposits.** On-chain state (token metadata, node records, contract storage) locks a per-byte deposit (placeholder about 0.07 ORAMA per KiB). On release, 99% is refunded to earnings and 1% is burned. This prices state growth.

Many other numbers (bonds, storage and relay parameters, caps) are labeled "G1 launch default" and are placeholders until a parameter model signs them off.

## 5. Governance and decentralization

**Bootstrap and hand-over.** Genesis names at least 30 validators (the production floor in code) with equal voting power and no stake, chosen from incentivized-testnet operators. Power then hands over: each validator's power is  $(1 - \lambda) \times \text{equal share} + \lambda \times \text{capped stake share}$ . Lambda never decreases and is the largest of three terms: bonded stake over a threshold (271,000 ORAMA, 5% of year-one emission), the elapsed fraction of a 365-epoch deadline, and its previous value. It reaches 1 within 365 epochs of genesis regardless of any vote, and committee-only seats then lapse. Lambda is held at 0.95 until the active set is large enough, and one block can raise published power by at most one third.

*Declared limit:* until lambda is 1, a permissioned committee chosen by human review secures the chain. That is weaker than a permissionless start, and the plan states it as a trust point. Half of each member's rewards are force-bonded so a double sign costs real stake.

**Capped power.** Power is linear in stake, capped at 5% per validator (3% above 60), with the excess redistributed. New stake ramps in over 30 epochs, and rewards are paid on capped power, so stake above the cap earns nothing extra. Slashing is 5% and removal for a double sign, 0.01% for downtime, with 21-day unbonding.

### Two houses.

- *Token house:* votes by stake. Delegated votes are capped at 3% of bonded stake per validator; direct votes are uncapped. Defaults: 40% quorum, 50% to pass, 7-day vote.
- *Operator house:* one vote per operator with at least 90 days of proven storage or relay service and a locked 1,000 ORAMA bond, at most 3 identities per /16 network and 5 per ASN. It can veto parameter changes (30% of the eligible house within 7 days) and must approve structural ones. Network and ASN are operator declarations the chain cannot verify. The code says so, and relies on bonds and a 14-day identity lock.

Nobody governs during bootstrap. The parameter tier opens when bonded stake reaches the threshold or lambda reaches 1, and the operator house has at least 21 eligible members. The structural tier (upgrades, split changes, relay reporters, the contract allow-list, development spends) opens only at lambda 1 with those 21 spanning at least 7 networks and 5 ASNs. Timelocks after passage: 14 days for parameters, 60 days for upgrades, 7 days for spends. There is no expedited path, and the timelock floors can be lengthened at genesis but never shortened.

**No admin keys, no pause, no freeze.** Every stock Cosmos module that expects a governance authority (upgrade, consensus, bank, staking, slashing, distribution) is given an authority address derived from a name that is never registered, so no key can sign for it.

`TestUnreachableAuthority_rejectsEveryAuthorityGatedMsg` proves each such message rejects any signer. `TestNoMessageReachesAnOssifiedField` fails if any governance message field touches the emission schedule, burn rule, privacy, or a freeze, halt, pause, multisig or authority. The SDK governance, mint, circuit-breaker and vesting modules are not wired.

**Upgrades.** A passed upgrade proposal schedules a plan after its 60-day timelock. At the plan height nodes halt unless the new binary is staged, and operators run cosmovisor with automatic download disabled. A bug fix is a coordinated halt-height release that validators holding two thirds of power choose to run, as in Bitcoin and Monero. Signed, reproducible releases with threshold signers are Designed, not yet built.

## 6. Privacy

---

**Shielded pool.** `x/shielded` uses Zcash's Orchard proof system (Halo 2, in the Ironwood pool version). The verifier is Rust called from Go, pinned to orchard 0.15.5 with only the corrected verifying key; the older unsound circuit is never built. Nodes only verify. Proving happens in the wallet. Upstream, the Orchard protocol and the Halo 2 proving system were reviewed in 2021, before Zcash's NU5 upgrade, by NCC Group ([report](#)) and QEDIT ([report](#)); neither found a serious or critical issue ([ECC summary](#)). That review covers the upstream circuits. It does not cover the corrected verifying key Orama pins or Orama's integration (turnstiles, nullifier store, unshield cap), which the in-house review funded by this round must cover.

**Mandatory shielding.** A bank send of `norama` from a user to a user is refused, and so is a send from a contract to a user. Users can pay contracts, and module accounts can pay. User-to-user ORAMA moves only as a signer-less `MsgShieldedTransfer`, whose fee comes out of the bundle. A node without both verifiers accepts no shielded bundle, so there user-to-user payment is impossible: it fails closed.

**Two verifiers.** Every bundle must be accepted by two verifiers: the linked library and a separately built, hash-pinned binary run out of process. A timeout or crash rejects the bundle. The honest limit: both call the same upstream crate, so a logic bug in that crate would be shared. An independent second implementation is an open item with audit budget reserved.

**Turnstile and cap.** Each pool vintage has a turnstile and never pays out more than went in. Net outflow is capped at the larger of 2% of the pool and a floor per rolling 24 hours, written in code. Unshielding goes only to targets the signer owns (their own delegation, node bond or fee balance) and is cryptographically bound to signer and target, so a copy in the mempool cannot be redirected. Excess requests queue and are served pro rata.

**Nullifiers.** Spent-note markers are stored in an append-only database outside the Merkle tree, with the app hash committing to a running accumulator. A crash between execution and commit replays to an identical hash (tested). A node whose database does not fold to the committed accumulator refuses to start.

**Censorship resistance.** Inclusion lists use CometBFT vote extensions. Each validator lists transactions it has held for 10 seconds or more (up to 32 KiB), and the proposer must place every valid listed transaction first or the block is rejected. A proposer therefore cannot censor a transaction that two thirds of power has seen. It switches on from a genesis height and is on in stagenet. No CometBFT crash test of this path has been run.

**Declared limits.** Contract and market payments reveal payer, amount and contract. A contract may issue a public IOU for ORAMA it holds. Designed, not yet built: multi-asset shielding, shielded delegation, the contract adapter, and verification timings on linux (the verification gas price is an unmeasured placeholder; one laptop measured 5.5 to 7 ms). Network-level privacy is partial: the transaction client can submit through a Tor proxy to a validator's onion service and never falls back to clearnet, but Orama's own relay network and validator onion services are Designed, not yet built.

## 7. Network services on chain

---

**Node registry (`x/nodes`).** Operators register nodes with roles (validator, storage, relay, exit, directory authority, archiver), signature-proven service keys, public endpoints and per-role bonds with 21-day unbonding. Slashing burns bond. New storage nodes can start on probation with no bond, earning from protocol-funded work, with a small record deposit built from early payouts.

**Storage (`x/storage`).** Three deal classes: private (encrypted pieces, with a per-slot key layer so a repair delegate can rebuild a replica without seeing plaintext), public pin (pinned on public IPFS), and archive (the chain's own history). Providers answer random challenges with proofs. Consecutive misses evict a slot and, from the second, slash. Payments split 90% provider, 5% burned, 5% archive fund. A subsidy from the storage ceiling starts at zero and ramps up as distinct operators increase, capped per operator.

**Relay (x/relay).** Relay operators are paid from the relay ceiling on epoch reports from a declared reporter set, capped per relay, operator and /16 network, with a 90% uptime minimum. Reporters change only through governance.

**Archive (x/archive).** Validators keep a window of blocks. Older history is bundled in 1,000-block ranges, attested by independent archivers and stored as archive deals.

**How operators earn.** Validators earn from the 60% pool by capped power, storage providers from the 25% pool and user deals, relays from the 10% pool. Everything lands in an earnings account that can fund the operator's own bonds and fees.

## 8. Smart contracts and assets

---

**CosmWasm.** Code upload is closed until a height written in genesis (about six months), then opens automatically. No vote can extend the closure, and before then only code hashes on a two-house allow-list may be stored. Genesis stores five standard contracts (CW20 token, CW721 NFT, escrow, fixed multisig, vesting), reproducibly built from pinned upstream commits. IBC and stargate queries are disabled. Contract storage is metered and locked as deposits.

**Orama bindings.** A contract can create and mint tokens, create collections and mint compressed NFTs, list and bid on the market, open a storage deal from its own funds, and pay a user through the earnings path. The shielded binding is a stub. Contracts calling Orama's cloud services (databases, functions) are Designed, not yet built.

**Tokens.** x/token issues denominations `factory/{creator}/{subdenom}`. Capabilities (mint authority, freeze, permanent delegate, transfer fee, non-transferable, pause) are chosen at creation and can only be renounced afterward. Creation burns 10 ORAMA and locks a metadata deposit. There is no on-chain "verified" registry.

**Compressed NFTs and market.** x/cnft keeps Merkle roots on chain, with leaves rebuildable from transaction bytes; a spike showed that events are not part of consensus, so they are never the source of truth. x/market runs listings and bids with creator royalties enforced in the market.

**Indexer.** A separate service indexes assets and ownership for wallets and explorers. The public website's explorer currently shows demonstration data.

## 9. How the chain connects to the Orama cloud

---

The chain and the cloud are separate by design. The chain is its own Go module and imports no cloud code. The cloud's module has no Cosmos, CometBFT or gogoprotobuf dependency (verified: zero matches in its dependency file). The cloud reaches the chain only over HTTP and RPC: its gateway exposes a read-only chain proxy, and the CLI signs chain transactions through the wallet agent. App requests never pass through the chain.

An app's backend (database, storage, functions, real-time) runs on an Orama cluster its owner controls. The chain carries what must be public and shared: money, the node registry, storage deals, tokens and contracts. A cluster may optionally register a public row on chain; registering joins no node to anything.

## 10. Status and what remains before mainnet

**Evidence.** In the most recent full fleet end-to-end run (2 October 2026, commit 2cf8b444), the eleven chain feature packages had 56 passing tests, no failures and 101 skipped, mostly because the run chain gives test accounts no bank balance. The shielded success path is covered by unit tests under the verifier build, not by that run. I could not confirm that the stagenet script's own smoke checks pass on the current commit.

| Module                       | Built                          | Tested                                   | On stagenet                       | Audited | Plan risk      |
|------------------------------|--------------------------------|------------------------------------------|-----------------------------------|---------|----------------|
| Emission, fees, power        | Yes                            | Unit, 4,000-epoch simulation, invariants | Yes                               | No      | High           |
| Houses (governance)          | Yes, tiers closed at bootstrap | Unit, ossified-field test                | Refusals only                     | No      | High           |
| Nodes, relay, archive        | Yes                            | Unit, fleet                              | Yes, partly                       | No      | Medium         |
| Storage                      | Yes                            | Unit, repair chaos test                  | Private deal flow in smoke script | No      | High           |
| Token, cNFT, market          | Yes                            | Unit, wallet flow                        | Refusals only                     | No      | Medium to High |
| CosmWasm and bindings        | Yes                            | Unit, app tests                          | Standard contracts stored         | No      | High           |
| Shielded pool                | Yes, two verifiers             | Unit, real-proof tests                   | Smoke script only                 | No      | Very high      |
| Inclusion lists              | Yes                            | Unit, app tests                          | Enabled                           | No      | High           |
| Confidential nodes, VPN gate | Library gates, not wired       | Unit                                     | No                                | No      | Outside scope  |

The chain has 58 transaction message types across nine custom modules, and ten modules answer an on-chain invariants query. The first chain commit is dated 27 September 2026; 149 commits have touched it since.

**Remaining before mainnet** (plan phase C17 and security programme):

- External audits of all modules, a separate zero-knowledge audit of the verifier, turnstile and cap, and of the power and inclusion code. None done.
- An incentivized testnet that scores operators for the bootstrap committee. Not run.
- The 30 to 150 validator benchmark, which sets the active-set size. Not run.
- A genesis ceremony and a rehearsed hard-fork playbook. Designed, not yet built.
- Signed, reproducible release process. Designed, not yet built.

- Orama's own relay network and validator onion services. Designed, not yet built.
- Funding of audits and bounties, which the plan assigns to the owner (O-D). The bounty policy defines scope and severity and promises no payout.

## 11. Open decisions

| Decision                                               | State                                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------------------------|
| Destination of the base fee                            | Open (Section 4). Code burns it today. Must be fixed in the genesis binary.           |
| Development ceiling (5%)                               | Plan records "keep" (O-A). May change with the fee decision.                          |
| Contracts holding ORAMA, public IOUs                   | Decided: allowed and declared (O-B).                                                  |
| How ORAMA reaches markets under mandatory shielding    | Open (O-C). Recommended: shielded exchange deposits plus in-chain exchange contracts. |
| Who funds pre-mainnet audits and bounties              | Open (O-D). Plan says the owner.                                                      |
| Active-set size (60 to 100)                            | Open until the benchmark runs.                                                        |
| Fee floor, token fee, bonds, storage and relay numbers | Placeholders pending a model and a testnet price.                                     |
| Coin type and wallet path                              | Placeholder 118.                                                                      |
| Block timing and block-denominated windows             | Recalibrate retention, anchor window and sunset (Section 3).                          |
| Independent second proof verifier                      | Open item, audit budget reserved.                                                     |